# NxtHelp / UniConnect — Complete Developer Architecture Guide

**Version:** 1.0
**Project:** NxtHelp (University Help Marketplace)
**Framework:** Django 4.2 (Python)
**Last Updated:** Auto-generated from source code analysis

---

## Table of Contents

1. Executive Summary
2. Technology Stack & Dependencies
3. Directory Structure
4. Project Configuration ( `nxthelp/` )
5. Entry Point ( `manage.py` )
6. Accounts App ( `accounts/` )
7. Work App ( `work/` )
8. Chat App ( `chat/` )
9. Frontend & Templates
10. Static Assets
11. Database Schema & Relationships
12. Complete URL Routing Map
13. Request / Response Lifecycle
14. Security Architecture
15. Deployment Configuration
16. Environment Variables Reference
17. Management Commands
18. Development Workflow Guide

---

# 1. Executive Summary

**NxtHelp** (branded as **UniConnect** in the UI) is a university-focused peer-to-peer help marketplace built with Django 4.2. It connects students, faculty, and staff who need assistance with others who can provide help. The platform supports:

- **Help Requests:** Any user can post a request with a title, description, budget, deadline, category, and optional image.
- **Applications:** Other users can apply to fulfill a request, proposing a message and optional price.
- **Chat System:** Isolated chat threads exist between a request poster and each applicant.
- **Notifications:** An in-app notification system alerts users to applications, acceptances, messages, and payments.
- **Payments:** Integrated Razorpay payment gateway for secure peer-to-peer transactions (INR currency).
- **Email Verification:** OTP-based email verification using Brevo (formerly Sendinblue) API for transactional emails.
- **Profiles:** Extended user profiles with profession, bio, skills, year, department, phone number, and UPI ID.

The architecture follows Django's MVT (Model-View-Template) pattern with three isolated apps: `accounts`, `work`, and `chat`.

---

# 2. Technology Stack & Dependencies

## Core Stack

| Component | Technology | Version | Purpose |
|---|---|---|---|
| Backend Framework | Django | 4.2 | MVT architecture, ORM, admin, security |
| WSGI Server | Gunicorn | >=21.2.0 | Production HTTP server |
| Database (Dev) | SQLite3 | Built-in | Local development |
| Database (Prod) | PostgreSQL | Via `psycopg2-binary` | Production database |
| DB Config | `dj-database-url` | >=2.1.0 | Parse `DATABASE_URL` env var |
| Static Files | WhiteNoise | >=6.5.0 | Serve static files in production |
| Environment | `python-dotenv` | >=1.0.0 | Load `.env` files locally |

## Third-Party Integrations

| Service | Library | Purpose |
|---|---|---|
| Razorpay | `razorpay>=1.4.1` | Payment gateway for INR transactions |
| Cloudinary | `cloudinary>=1.44.2` , `django-cloudinary-storage>=0.3.0` | Media file storage (request images) |
| Brevo SMTP | `urllib` (stdlib) + Brevo REST API | Transactional OTP emails |
| Tailwind CSS | CDN | Utility-first CSS framework |
| HTMX | CDN | AJAX interactions without heavy JS |
| FontAwesome | CDN | Iconography |

## Frontend Stack

- **HTML Templates:** Django Template Language (DTL)
- **CSS:** Custom design system in `static/css/style.css` + Tailwind CSS via CDN
- **JavaScript:** Vanilla JS + HTMX for AJAX
- **Design Philosophy:** Clean, minimal, mobile-first responsive design with glass-morphism and subtle animations

---

# 3. Directory Structure

```
nxthelp/
├── manage.py                   # Django CLI entry point
├── requirements.txt            # Python dependencies
├── railway.toml                # Railway.app deployment config
├── Project_Overview.md         # High-level project overview
├── Security_Audit_Report.md    # Security audit findings
│
├── nxthelp/                    # Project configuration (root package)
│   ├── __init__.py
│   ├── settings.py             # All Django settings
│   ├── urls.py                 # Root URL router
│   ├── wsgi.py                 # WSGI entry point
│   └── asgi.py                 # ASGI entry point
│
├── accounts/                   # App: Identity, Auth, Profiles
│   ├── models.py               # Profile, OTPToken
│   ├── views.py                # Auth & profile views
│   ├── urls.py                 # App URL routes
│   ├── forms.py                # Registration, login, profile forms
│   ├── admin.py                # Django admin config
│   ├── backends.py             # Custom auth backend
│   ├── signals.py              # Auto-create Profile on User creation
│   ├── apps.py                 # App configuration
│   └── templates/accounts/     # Auth templates
│
├── work/                       # App: Core business logic
│   ├── models.py               # HelpRequest, Application, Category, Notification, Payment
│   ├── views.py                # Dashboard, requests, applications, payments
│   ├── urls.py                 # App URL routes
│   ├── forms.py                # HelpRequestForm, ApplicationForm
│   ├── admin.py                # Django admin config
│   ├── apps.py                 # App configuration
│   ├── management/commands/    # Custom CLI commands
│   │   └── seed_categories.py  # Seed initial categories
│   └── templates/work/         # Work templates
│
├── chat/                       # App: Messaging system
│   ├── models.py               # ChatMessage
│   ├── views.py                # Chat room, send/fetch messages
│   ├── urls.py                 # App URL routes
│   ├── admin.py                # Django admin config
│   └── templates/chat/         # Chat templates
│
├── templates/                  # Global templates
│   └── base.html               # Master layout template
│
└── static/
```

```
    └── css/
        └── style.css           # Custom CSS design system
```

---

# 4. Project Configuration ( `nxthelp/` )

## 4.1 `nxthelp/settings.py`

**Purpose:** Central configuration file for the entire Django project. Controls security, database, installed apps, middleware, static files, email, logging, and third-party integrations.

**Key Sections:**

### Security Configuration

- `SECRET_KEY` : Loaded from `DJANGO_SECRET_KEY` environment variable. Falls back to a dummy key during `collectstatic` or `makemigrations` to prevent build-time crashes.
- `DEBUG` : Controlled by `DJANGO_DEBUG` env var.
- `ALLOWED_HOSTS` : Dynamically built from `DJANGO_ALLOWED_HOSTS` env var plus default domains: `.onrender.com` , `.ondigitalocean.app` , `.up.railway.app` , `nxthelp.in` , `localhost` , `127.0.0.1` .
- `CSRF_TRUSTED_ORIGINS` : Dynamically built with HTTPS origins for known platforms.
- **Production Security Headers** (active when `DEBUG=False` ):
- `SECURE_PROXY_SSL_HEADER` : Trust Railway's proxy
- `SECURE_HSTS_SECONDS` : 1 year (31536000)
- `SECURE_HSTS_INCLUDE_SUBDOMAINS` : True
- `SECURE_HSTS_PRELOAD` : True
- `SECURE_SSL_REDIRECT` : True
- `SESSION_COOKIE_SECURE` , `CSRF_COOKIE_SECURE` : True
- `SESSION_COOKIE_HTTPONLY` , `SESSION_COOKIE_SAMESITE` : Lax
- `SECURE_BROWSER_XSS_FILTER` , `X_FRAME_OPTIONS = 'DENY'` , `SECURE_CONTENT_TYPE_NOSNIFF` : Always active.

### Installed Apps

```
INSTALLED_APPS = [
    # Django Core
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Local Apps
    'accounts.apps.AccountsConfig',
    'work.apps.WorkConfig',
    'chat.apps.ChatConfig',
    # Third-Party
    'cloudinary_storage',
    'cloudinary',
]
```

### Middleware Stack

```
1. django.middleware.security.SecurityMiddleware
2. whitenoise.middleware.WhiteNoiseMiddleware      # Static file serving
3. django.contrib.sessions.middleware.SessionMiddleware
4. django.middleware.common.CommonMiddleware
5. django.middleware.csrf.CsrfViewMiddleware
6. django.contrib.auth.middleware.AuthenticationMiddleware
7. django.contrib.messages.middleware.MessageMiddleware
8. django.middleware.clickjacking.XFrameOptionsMiddleware
```

### Database Configuration

- **Logic:** If `makemigrations` is in `sys.argv`, use SQLite. Otherwise, if `DATABASE_URL` env var exists, parse it with `dj_database_url`. Fall back to SQLite.
- **Connection Persistence:** `conn_max_age=600` for PostgreSQL.

### Cloudinary Media Storage

- Configured with `CLOUDINARY_CLOUD_NAME`, `API_KEY`, `API_SECRET` env vars.
- Only activates if all three are present.
- Sets `DEFAULT_FILE_STORAGE` to `cloudinary_storage.storage.MediaCloudinaryStorage`.

### Authentication

- `LOGIN_URL = 'login'`

- `LOGIN_REDIRECT_URL = 'dashboard'`
- `LOGOUT_REDIRECT_URL = 'landing'`
- `AUTHENTICATION_BACKENDS` : Custom `EmailOrUsernameModelBackend` first, then Django's default.

### Razorpay Configuration

- `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` from env vars.
- `RAZORPAY_CURRENCY = 'INR'`

### Email Configuration

- `EMAIL_BACKEND` : Defaults to SMTP, overridable via env var.
- `EMAIL_HOST` : Defaults to `smtp.gmail.com`
- `EMAIL_PORT` : 587
- `EMAIL_USE_TLS` : True
- Credentials from `EMAIL_HOST_USER` and `EMAIL_HOST_PASSWORD` env vars.

### Logging

- Root logger: WARNING level to console.
- `django.request` : ERROR level.
- `accounts` : INFO level.
- Format: `[{levelname}] {asctime} {name}: {message}`

## 4.2 `nxthelp/urls.py`

**Purpose:** Root URL configuration. Routes incoming requests to the appropriate app-level URL modules.

**URL Patterns:**

| Path | Included App |
|---|---|
| `admin/` | Django Admin |
| `''` (root) | `accounts.urls` |
| `''` (root) | `work.urls` |
| `''` (root) | `chat.urls` |

All three apps share the root URL namespace. Route resolution depends on the order of inclusion and specificity of paths within each app's `urls.py` .

### 4.3 `nxthelp/wsgi.py`

**Purpose:** WSGI application entry point for production servers (Gunicorn).
- Sets `DJANGO_SETTINGS_MODULE` to `nxthelp.settings`.
- Exposes `application = get_wsgi_application()`.

### 4.4 `nxthelp/asgi.py`

**Purpose:** ASGI application entry point (for future async/WebSocket support).
- Currently mirrors WSGI setup with `get_asgi_application()`.

---

## 5. Entry Point (`manage.py`)

**Purpose:** Standard Django command-line utility.

**Workflow:**
1. Sets `DJANGO_SETTINGS_MODULE` environment variable to `nxthelp.settings`.
2. Imports `execute_from_command_line` from Django.
3. Executes the command passed via CLI arguments (`runserver`, `migrate`, `shell`, etc.).

**Key Usage:**

```
python manage.py runserver          # Development server
python manage.py migrate            # Apply migrations
python manage.py makemigrations     # Create migrations
python manage.py collectstatic      # Gather static files
python manage.py seed_categories    # Custom: seed categories
```

---

## 6. Accounts App (`accounts/`)

**Purpose:** Handles user identity, authentication, email verification, and profile management.

### 6.1 `accounts/models.py`

**Models:**

`Profile`

Extends Django's built-in `User` with university-specific fields.

| Field | Type | Description |
|---|---|---|
| `user` | OneToOneField(User) | Linked user account |
| `profession` | CharField(max=150) | Role at university (e.g., "3rd Year CSE Student") |
| `bio` | TextField(max=500) | User biography |
| `skills` | CharField(max=300) | Comma-separated skills list |
| `year` | CharField(choices) | 1-5, Alumni |
| `department` | CharField(max=100) | Academic department |
| `avatar_color` | CharField(max=7, default='#6C63FF') | Random color for avatar initials |
| `phone_number` | CharField(max=15) | Phone for manual payments |
| `upi_id` | CharField(max=100) | UPI ID for payments |
| `created_at` | DateTimeField(auto_now_add) | Profile creation timestamp |

**Methods:**
- `get_skills_list()` : Parses comma-separated skills into a Python list.
- `@property display_name` : Returns `full_name` if available, otherwise `username`.
- `@property initials` : Returns uppercase initials (e.g., "John Doe" → "JD").

### `OTPToken`

Manages 6-digit email verification codes.

| Field | Type | Description |
|---|---|---|
| `user` | OneToOneField(User) | Linked user |
| `otp_code` | CharField(max=6) | 6-digit numeric code |
| `created_at` | DateTimeField(auto_now_add) | Generation timestamp |

**Methods:**
- `is_expired()` : Returns `True` if more than 10 minutes have passed since creation.

## 6.2 `accounts/views.py`

**Helper Functions:**

### `_send_otp_email(request, user)`

**Purpose:** Generates a 6-digit OTP and sends it via Brevo API.
**Workflow:**
1. Deletes any existing OTP token for the user.
2. Generates random 6-digit code.
3. Creates `OTPToken` record.
4. Renders HTML email template ( `accounts/email_verification.html` ).
5. Sends via Brevo REST API ( `api.brevo.com/v3/smtp/email` ) using `BREVO_API_KEY` env var.
6. Returns `True` on success, `False` on failure (logs errors).

**Views:**

### `landing_page(request)`

**Route:** `/` (name: `landing` )
**Purpose:** Public landing page. Shows platform stats (total users, posted requests, resolved requests). Redirects authenticated users to dashboard.

### `register_view(request)`

**Route:** `/register/` (name: `register` )
**Purpose:** User registration with OTP verification.
**Workflow:**
1. Validates `UserRegisterForm` .
2. Creates user with `is_active=False` .
3. Saves profession to profile.
4. Sends OTP email via `_send_otp_email` .
5. Stores email in session and redirects to `verify_otp` .

### `verify_otp_view(request)`

**Route:** `/verify-otp/` (name: `verify_otp` )
**Purpose:** Validates the 6-digit OTP to activate the account.
**Workflow:**
1. Retrieves email from session.
2. Finds inactive user by email.
3. Enforces max 5 attempts per session (tracked via `otp_attempts_{user.pk}` ).
4. Checks OTP validity and expiry (10-minute window).
5. On success: activates user, deletes OTP, logs in user, redirects to dashboard.
6. On expiry: deletes old OTP, sends new one, warns user.

### `resend_verification(request)`

**Route:** `/resend-verification/` (name: `resend_verification`)
**Purpose:** Resends OTP with rate limiting (60-second cooldown).
**Security:** Prevents email enumeration by showing generic success message even if email not found.

### `login_view(request)`

**Route:** `/login/` (name: `login`)
**Purpose:** Authenticates users via email or username.
**Workflow:**
1. Uses `EmailOrUsernameAuthForm`.
2. If user is inactive, redirects to OTP verification.
3. Validates `next` parameter to prevent open redirect attacks using `url_has_allowed_host_and_scheme`.

### `logout_view(request)`

**Route:** `/logout/` (name: `logout`)
**Purpose:** POST-only logout to prevent CSRF logout attacks.

### `profile_view(request, username=None)`

**Route:** `/profile/` (name: `profile`), `/profile/<username>/` (name: `user_profile`)
**Purpose:** Displays user profile with recent posted requests and helped requests.

### `edit_profile(request)`

**Route:** `/profile/edit/` (name: `edit_profile`)
**Purpose:** Allows users to update their profile and email.
**Workflow:**
1. Validates `ProfileUpdateForm`.
2. If email changes: deactivates account, sends new OTP, logs out user, redirects to verification.

## 6.3 `accounts/urls.py`

| Path | View | Name |
|---|---|---|
| `''` | `landing_page` | `landing` |
| `register/` | `register_view` | `register` |
| `login/` | `login_view` | `login` |
| `logout/` | `logout_view` | `logout` |
| `verify-otp/` | `verify_otp_view` | `verify_otp` |
| `resend-verification/` | `resend_verification` | `resend_verification` |
| `profile/` | `profile_view` | `profile` |
| `profile/edit/` | `edit_profile` | `edit_profile` |
| `profile/<str:username>/` | `profile_view` | `user_profile` |

## 6.4 `accounts/forms.py`

### `EmailOrUsernameAuthForm(AuthenticationForm)`

- Overrides `confirm_login_allowed()` to allow inactive users to pass validation.
- This prevents timing-based enumeration and lets the view handle inactive accounts uniformly.

### `UserRegisterForm(UserCreationForm)`

**Fields:** `username`, `first_name`, `last_name`, `email`, `password1`, `password2`, `profession`
- Custom widgets with CSS classes ( `form-input` ).
- `clean_email()` : Enforces unique email addresses.
- `save()` : Creates user, sets profile profession.

### `ProfileUpdateForm(forms.ModelForm)`

**Model:** `Profile`
**Fields:** `profession`, `bio`, `skills`, `year`, `department`, `phone_number`, `upi_id`
- Also includes `first_name`, `last_name`, `email` (from User model).
- `clean_email()` : Excludes current user to allow keeping same email.
- `save()` : Updates both `Profile` and `User` models.

### 6.5 `accounts/backends.py`

`EmailOrUsernameModelBackend(ModelBackend)`

**Purpose:** Custom authentication backend allowing login with either username or email.
**Workflow:**
1. Receives `username` parameter (can be email or username).
2. Queries `User` with `Q(username__iexact=username) | Q(email__iexact=username)`.
3. Returns user only if password is correct and `user_can_authenticate()` passes.

### 6.6 `accounts/signals.py`

**Signals:**
- `post_save` on `User` → `create_profile()`: Auto-creates a `Profile` with a random avatar color when a new user is created.
- `post_save` on `User` → `save_profile()`: Auto-saves the related profile whenever the user is saved.

### 6.7 `accounts/admin.py`

- Registers `Profile` model in Django Admin.
- `list_display`: `user`, `profession`, `department`, `year`, `created_at`
- `list_filter`: `year`, `department`
- `search_fields`: `user__username`, `user__first_name`, `user__last_name`, `profession`

### 6.8 `accounts/apps.py`

- `AccountsConfig` class with `ready()` method.
- Imports `accounts.signals` to ensure signal handlers are registered at app startup.

---

## 7. Work App (`work/`)

**Purpose:** Core business logic. Manages help requests, applications, notifications, categories, and payments.

### 7.1 `work/models.py`

`Category`

Categorizes help requests.

| Field | Type | Description |
|---|---|---|
| `name` | CharField(max=100, unique) | Category name |
| `slug` | SlugField(unique) | URL-friendly identifier |
| `icon` | CharField(max=50) | FontAwesome icon class |
| `color` | CharField(max=7) | Hex color code |
| `description` | TextField | Category description |

### `HelpRequest`

Central entity of the platform.

| Field | Type | Description |
|---|---|---|
| `title` | CharField(max=200) | Request title |
| `description` | TextField | Detailed description |
| `posted_by` | ForeignKey(User) | Request author |
| `category` | ForeignKey(Category, null=True) | Optional category |
| `urgency` | CharField(choices) | low, medium, high |
| `status` | CharField(choices) | open, in_progress, completed, resolved, closed |
| `request_type` | CharField(choices) | personal, academic, non_academic |
| `target_year` | CharField(choices) | all, 1, 2, 3, 4, 5 |
| `budget` | CharField(max=100) | Budget text (e.g., "₹500") |
| `deadline` | DateField(null=True) | Optional deadline |
| `image` | CloudinaryField | Optional image attachment |
| `selected_helper` | ForeignKey(User, null=True) | Assigned helper |

**Properties:**
- `application_count` : Count of related applications.
- `is_open` : True if status == 'open'.
- `is_overdue` : True if deadline passed and status is open/in_progress.

- `days_until_deadline` : Integer days until deadline.
- `time_since_posted` : Human-readable relative time (e.g., "2h ago", "3d ago").
- `status_css_class` : CSS-friendly status string.

### `Application`

Represents a user's application to help with a request.

| Field | Type | Description |
|---|---|---|
| `help_request` | ForeignKey(HelpRequest) | Parent request |
| `applicant` | ForeignKey(User) | User applying to help |
| `message` | TextField | Application message |
| `proposed_budget` | CharField(max=100) | Optional proposed price |
| `status` | CharField(choices) | pending, accepted, rejected, withdrawn, completed |

**Constraint:** `unique_together = ['help_request', 'applicant']` — one application per user per request.

### `Notification`

In-app notification system.

| Field | Type | Description |
|---|---|---|
| `recipient` | ForeignKey(User) | Notification target |
| `notification_type` | CharField(choices) | Type of event |
| `title` | CharField(max=200) | Short title |
| `message` | TextField | Detailed message |
| `link` | CharField(max=200) | Optional redirect URL |
| `is_read` | BooleanField(default=False) | Read status |

**Types:** `new_application` , `application_accepted` , `application_rejected` , `request_resolved` , `new_message` , `work_completed` , `payment_initiated` , `payment_received` .

### `Payment`

Payment transaction record.

| Field | Type | Description |
|---|---|---|
| `transaction_id` | UUIDField(unique) | Unique transaction identifier |
| `help_request` | ForeignKey(HelpRequest) | Related request |
| `payer` | ForeignKey(User) | Person paying |
| `payee` | ForeignKey(User) | Person receiving payment |
| `amount` | DecimalField(max=10, decimal_places=2) | Transaction amount |
| `payment_method` | CharField(choices) | razorpay, upi, phone |
| `payment_address` | CharField(max=100) | UPI ID or phone number |
| `status` | CharField(choices) | created, pending, completed, failed |
| `note` | TextField | Optional payment note |
| `razorpay_order_id` | CharField | Razorpay order ID |
| `razorpay_payment_id` | CharField | Razorpay payment ID |
| `razorpay_signature` | CharField(max=255) | Razorpay signature for verification |

**Property:**
- `masked_address` : Privacy-aware masking of UPI/phone numbers.

## 7.2 `work/views.py`

### Razorpay Client ( `get_razorpay_client()` )

- Lazy-initialized singleton.
- Checks for `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` .
- Raises clear error if keys are missing.

### `dashboard(request)`

**Route:** `/dashboard/` (name: `dashboard` )
**Purpose:** Unified dashboard for authenticated users.

**Context:**
- My requests (posted) with counts (total, open, in_progress, resolved)
- My applications with counts (total, pending, accepted)
- Pending applications on my requests
- Available open requests (excluding own and already applied)
- Upcoming deadlines
- Recent unread notifications
- Unread chat message count

### `create_help_request(request)`

**Route:** `/request/new/` (name: `create_help_request`)
**Purpose:** Create a new help request.
**Form:** `HelpRequestForm`
**Workflow:** Saves form with `posted_by = request.user`.

### `help_request_detail(request, pk)`

**Route:** `/request/<int:pk>/` (name: `help_request_detail`)
**Purpose:** View request details.
**Context Differences:**
- **Owner (`is_owner=True`):** Sees all applications with last chat message and unread count per application.
- **Non-Owner:** Sees their existing application (if any) or an application form (if open and not applied).

### `browse_requests(request)`

**Route:** `/browse/` (name: `browse_requests`)
**Purpose:** Browse all open help requests with filtering.
**Filters (GET params):**
- `category`: Filter by category slug
- `urgency`: low, medium, high
- `request_type`: personal, academic, non_academic
- `target_year`: Specific year or 'all'
- `q`: Text search on title and description

### `my_requests(request)`

**Route:** `/my-requests/` (name: `my_requests`)
**Purpose:** List current user's posted requests with optional `status` filter.

### `apply_to_help(request, pk)`

**Route:** `/request/<int:pk>/apply/` (name: `apply_to_help`)
**Purpose:** Submit application to a help request.
**Guards:**
- Cannot apply to own request.
- Cannot apply if request is not open.
- Cannot apply twice.

**Workflow:**
1. Validates `ApplicationForm`.
2. Creates `Application` with status `pending`.
3. Creates `Notification` for the request poster.
4. Redirects to chat room for immediate communication.

### `withdraw_application(request, pk)`

**Route:** `/application/<int:pk>/withdraw/` (name: `withdraw_application`)
**Purpose:** Withdraw an application.
**Guard:** Only the applicant can withdraw.
**Side Effect:** Notifies the request poster.

### `resolve_request(request, pk)`

**Route:** `/request/<int:pk>/resolve/` (name: `resolve_request`)
**Purpose:** Mark request as resolved.
**Guard:** Only the poster.
**Side Effect:** Notifies all non-withdrawn applicants.

### `close_request(request, pk)`

**Route:** `/request/<int:pk>/close/` (name: `close_request`)
**Purpose:** Close a request (status = 'closed').

### `my_applications(request)`

**Route:** `/my-applications/` (name: `my_applications`)
**Purpose:** List user's sent applications with optional `status` filter.

### `payment_page(request, pk)`

**Route:** `/request/<int:pk>/payment/` (name: `payment_page`)
**Purpose:** Payment interface with Razorpay checkout.
**Guards:** Only poster, request must be in_progress or completed, helper must be assigned.

**Amount Calculation:**
1. Attempts to extract numeric value from accepted application's `proposed_budget`.
2. Falls back to `help_request.budget`.
3. Passes amount to template for Razorpay checkout initialization.

### `create_razorpay_order(request, pk)`

**Route:** `/request/<int:pk>/payment/create-order/` (name: `create_razorpay_order`)
**Method:** POST (AJAX/JSON)
**Purpose:** Creates a Razorpay order server-side.
**Security:** Amount is calculated on the backend from the accepted application/request budget — never trusts client-provided amount.
**Workflow:**
1. Parses JSON body for optional note.
2. Calculates `backend_amount` from `proposed_budget` or `budget` using regex extraction.
3. Converts to paise (INR smallest unit).
4. Calls Razorpay API to create order.
5. Creates `Payment` record with status `created`.
6. Returns JSON with `order_id`, `amount`, `currency`, `payment_pk`.

### `confirm_payment(request, pk)`

**Route:** `/request/<int:pk>/payment/confirm/` (name: `confirm_payment`)
**Method:** POST
**Purpose:** Verifies Razorpay payment signature and completes transaction.
**Workflow:**
1. Receives `razorpay_payment_id`, `razorpay_order_id`, `razorpay_signature` from POST.
2. Finds matching `Payment` record.
3. Calls Razorpay's `verify_payment_signature()` utility.
4. On success:
- Updates payment status to `completed`.
- Updates request status to `resolved`.
- Notifies helper.
5. On failure: Marks payment as `failed`.

### `payment_receipt(request, pk)`

**Route:** `/request/<int:pk>/payment/receipt/` (name: `payment_receipt`)
**Purpose:** Display receipt for completed payment.
**Guard:** Only payer and payee can view.

### `notifications_view(request)`

**Route:** `/notifications/` (name: `notifications`)
**Purpose:** List all notifications for current user.

### `mark_notification_read(request, pk)`

**Route:** `/notifications/<int:pk>/read/` (name: `mark_notification_read`)
**Purpose:** Mark single notification as read and redirect to its link (if any).

### `mark_all_read(request)`

**Route:** `/notifications/mark-all-read/` (name: `mark_all_read`)
**Purpose:** Bulk mark all unread notifications as read.

### `notification_count(request)`

**Purpose:** Context processor (referenced in `settings.py`).
**Returns:** `{'unread_notification_count': int}` for the authenticated user.

## 7.3 `work/urls.py`

| Path | View | Name |
|---|---|---|
| `dashboard/` | `dashboard` | `dashboard` |
| `request/new/` | `create_help_request` | `create_help_request` |
| `request/<int:pk>/` | `help_request_detail` | `help_request_detail` |
| `browse/` | `browse_requests` | `browse_requests` |
| `my-requests/` | `my_requests` | `my_requests` |
| `request/<int:pk>/apply/` | `apply_to_help` | `apply_to_help` |
| `application/<int:pk>/withdraw/` | `withdraw_application` | `withdraw_application` |
| `my-applications/` | `my_applications` | `my_applications` |
| `request/<int:pk>/resolve/` | `resolve_request` | `resolve_request` |
| `request/<int:pk>/close/` | `close_request` | `close_request` |
| `request/<int:pk>/payment/` | `payment_page` | `payment_page` |
| `request/<int:pk>/payment/create-order/` | `create_razorpay_order` | `create_razorpay_order` |
| `request/<int:pk>/payment/confirm/` | `confirm_payment` | `confirm_payment` |
| `request/<int:pk>/payment/receipt/` | `payment_receipt` | `payment_receipt` |
| `notifications/` | `notifications_view` | `notifications` |
| `notifications/<int:pk>/read/` | `mark_notification_read` | `mark_notification_read` |
| `notifications/mark-all-read/` | `mark_all_read` | `mark_all_read` |

## 7.4 `work/forms.py`

### `HelpRequestForm(forms.ModelForm)`

**Model:** `HelpRequest`
**Fields:** `title`, `description`, `urgency`, `request_type`, `target_year`, `budget`, `deadline`, `image`
**Widgets:** Custom CSS classes ( `form-input`, `form-textarea` ). Date input uses HTML5 `type="date"`. Image

input accepts `image/*`.
**Customization:** `deadline`, `budget`, and `image` are optional.

`ApplicationForm(forms.ModelForm)`

**Model:** `Application`
**Fields:** `message`, `proposed_budget`
**Widgets:** Textarea for message, text input for budget.
**Customization:** `proposed_budget` is optional.

### 7.5 `work/admin.py`

Registers all `work` models in Django Admin:
- **Category:** `list_display`, `prepopulated_fields={'slug': ('name',)}`
- **HelpRequest:** `list_display`, `list_filter`, `search_fields`, `raw_id_fields`
- **Application:** `list_display`, `list_filter`, `raw_id_fields`
- **Notification:** `list_display`, `list_filter`, `raw_id_fields`
- **Payment:** `list_display`, `list_filter`, `search_fields`, `raw_id_fields`, `readonly_fields=('transaction_id',)`

### 7.6 `work/management/commands/seed_categories.py`

**Command:** `python manage.py seed_categories`
**Purpose:** Idempotently seeds 8 initial categories:
1. Programming & CS
2. Mathematics
3. Science & Engineering
4. Writing & Research
5. Design & Creative
6. Languages
7. Career & Internships
8. Other

Uses `get_or_create(slug=...)` to avoid duplicates.

---

## 8. Chat App (`chat/`)

**Purpose:** Provides isolated messaging between request posters and applicants.

## 8.1 `chat/models.py`

### `ChatMessage`

| Field | Type | Description |
|---|---|---|
| `help_request` | ForeignKey(HelpRequest) | Parent request |
| `sender` | ForeignKey(User) | Message author |
| `application` | ForeignKey(Application, null=True) | Specific application thread |
| `content` | TextField | Message text |
| `created_at` | DateTimeField(auto_now_add) | Timestamp |
| `is_read` | BooleanField(default=False) | Read status |

**Ordering:** By `created_at` (oldest first) for chronological display.

## 8.2 `chat/views.py`

### `chat_room(request, pk, app_pk)`

**Route:** `/request/<int:pk>/chat/<int:app_pk>/` (name: `chat_room`)
**Purpose:** Render chat thread.
**Guard:** Only poster and applicant can access.
**Workflow:**
1. Marks unread messages as read.
2. Fetches all messages for the application.
3. Determines `other_user` for the chat header.

### `send_message(request, pk, app_pk)`

**Route:** `/request/<int:pk>/chat/<int:app_pk>/send/` (name: `send_message`)
**Method:** POST
**Purpose:** Send a new message.
**Workflow:**
1. Validates non-empty content.
2. Creates `ChatMessage`.
3. Creates `Notification` for the recipient.
4. If AJAX request: returns JSON with message metadata (id, content, sender info, time).
5. If normal POST: redirects to chat room.

### `fetch_messages(request, pk, app_pk)`

**Route:** `/request/<int:pk>/chat/<int:app_pk>/fetch/` (name: `fetch_messages`)
**Method:** GET (AJAX)
**Purpose:** Poll for new messages.
**Query Param:** `last_id` — fetches messages with ID > last_id.
**Workflow:**
1. Validates access.
2. Fetches new messages.
3. Marks them as read.
4. Returns JSON array of message objects.

### `my_chats(request)`

**Route:** `/my-chats/` (name: `my_chats`)
**Purpose:** List all active chat threads grouped by help request.
**Workflow:**
1. Finds all applications where user is involved (as poster or applicant) and has at least one chat message.
2. Groups by `help_request`.
3. For each thread, computes `other_user`, `last_message`, and `unread_count`.

## 8.3 `chat/urls.py`

| Path | View | Name |
|---|---|---|
| `request/<int:pk>/chat/<int:app_pk>/` | `chat_room` | `chat_room` |
| `request/<int:pk>/chat/<int:app_pk>/send/` | `send_message` | `send_message` |
| `request/<int:pk>/chat/<int:app_pk>/fetch/` | `fetch_messages` | `fetch_messages` |
| `my-chats/` | `my_chats` | `my_chats` |

## 8.4 `chat/admin.py`

- Registers `ChatMessage` with `list_display`, `list_filter=('is_read',)`, `raw_id_fields`.

---

# 9. Frontend & Templates

## 9.1 `templates/base.html`

**Purpose:** Master layout template inherited by all pages.

**Features:**
- **Meta:** Responsive viewport, SEO description.
- **Fonts:** Inter (body) + Plus Jakarta Sans (headings) via Google Fonts.
- **CSS:** FontAwesome 6, custom `style.css`, Tailwind CSS CDN with custom config.
- **Tailwind Config:** Custom `brand` and `surface` color palettes, custom shadows (`glass`, `soft`, `glow`), custom animations.
- **HTMX:** `htmx.org@1.9.11` for AJAX page transitions.
- **Background:** Fixed position animated gradient orbs (`float`, `float-delayed`).

**Authenticated Navigation:**
- **Desktop Navbar:** Glass-morphism sticky header with brand logo, dashboard/browse links, "Post Request" CTA, notification bell with badge, avatar dropdown.
- **Avatar Dropdown:** Profile, My Requests, My Applications, Chats, Settings, Logout.
- **Mobile Bottom Nav:** Fixed bottom bar with Home, Browse, Post (+), Chats, Profile.

**Toast Messages:**
- Fixed top-right container.
- Auto-dismiss after 4.5 seconds.
- Color-coded by message tags (success, error, warning, info).

**JavaScript:**
- HTMX CSRF token injection.
- Avatar dropdown toggle with outside-click close.
- Toast auto-dismiss.
- `initDynamicElements()` runs on initial load and HTMX content swaps.

## 9.2 Template Hierarchy

**Accounts Templates (`accounts/templates/accounts/`):**
- `landing.html` — Public landing page
- `login.html` — Login form
- `register.html` — Registration form
- `verify_otp.html` — OTP input
- `email_verification.html` — Email HTML template (also used by Brevo)
- `profile.html` — User profile display
- `edit_profile.html` — Profile edit form

**Work Templates ( `work/templates/work/` ):**
- `dashboard.html` — Main dashboard
- `browse_requests.html` — Request listing with filters
- `request_detail.html` — Request detail + applications
- `create_request.html` — New request form
- `apply.html` — Application form
- `my_requests.html` — User's posted requests
- `my_applications.html` — User's sent applications
- `notifications.html` — Notification list
- `payment.html` — Razorpay checkout page
- `payment_receipt.html` — Payment receipt

**Chat Templates ( `chat/templates/chat/` ):**
- `chat.html` — Chat room interface
- `my_chats.html` — Chat thread listing

---

# 10. Static Assets

## 10.1 `static/css/style.css`

**Purpose:** Custom CSS design system that complements Tailwind CSS.

**Key Sections:**
- **CSS Variables:** `--primary: #7c3aed` , `--green: #16a34a` , `--red: #ef4444` , `--amber: #f59e0b` , `--slate-*` scale, `--radius: 12px` , `--shadow` .
- **Reset & Base:** Border-box, margin/padding reset, Inter font family.
- **Navbar:** `.nxt-navbar` (legacy class, but overridden by Tailwind in `base.html` ).
- **Mobile Navigation:** Fixed bottom bar, hidden on desktop ( `@media (min-width: 992px)` ).
- **Avatars:** `.avatar-sm` , `.avatar-md` , `.avatar-lg` , `.avatar-xl` with circular styling.
- **Buttons:** `.btn` , `.btn-primary` , `.btn-secondary` , `.btn-success` , `.btn-danger` , `.btn-ghost` , `.btn-outline-white` .
- **Forms:** `.form-group` , `.form-label` , `.form-input` , `.form-textarea` , `.form-error` . Focus states with purple border and ring.
- **Cards:** `.card` , `.sidebar-card` , `.auth-card` . Hover transitions.
- **Stats:** `.stats-grid` , `.stat-card` , `.stat-icon` (color variants: purple, green, yellow, blue, red).
- **Request Cards:** `.requests-grid` (responsive 1→2→3 columns), `.request-card` with hover lift and glow.
- **Badges:** `.badge-*` for statuses (open, in-progress, resolved, closed, pending, accepted, rejected, withdrawn, urgency levels).
- **Chat UI:** `.chat-container` (full viewport height), `.chat-bubble.mine` (purple, right), `.chat-bubble.theirs` (gray, left), `.chat-input-area` .

- **Toasts:** `.toast-container-custom` (fixed top-right), `.toast-custom` with slide-in animation.
- **Auth Pages:** `.auth-page` with gradient background, `.auth-card`.

---

# 11. Database Schema & Relationships

## Entity Relationship Overview

```
User (Django built-in)
│
├─ OneToOne ──> Profile (accounts)
│
├─ ForeignKey ──> HelpRequest (work) [posted_by]
├─ ForeignKey ──> HelpRequest (work) [selected_helper]
├─ ForeignKey ──> Application (work) [applicant]
├─ ForeignKey ──> Notification (work) [recipient]
├─ ForeignKey ──> Payment (work) [payer]
├─ ForeignKey ──> Payment (work) [payee]
├─ ForeignKey ──> ChatMessage (chat) [sender]
└─ OneToOne ──> OTPToken (accounts) [user]

Category (work)
└─ ForeignKey ──> HelpRequest (work) [category]

HelpRequest (work)
├─ ForeignKey ──> Application (work) [help_request]
├─ ForeignKey ──> ChatMessage (chat) [help_request]
└─ ForeignKey ──> Payment (work) [help_request]

Application (work)
└─ ForeignKey ──> ChatMessage (chat) [application]
```

## Migration History

**Accounts:**

1. `0001_initial.py` — Creates `Profile`
2. `0002_profile_phone_number_profile_upi_id.py` — Adds payment fields
3. `0003_emailverificationtoken.py` — Adds email token model
4. `0004_otptoken_delete_emailverificationtoken.py` — Replaces with OTP model

**Work:**

1. `0001_initial.py` — Creates `Category`, `HelpRequest`, `Application`, `Notification`
2. `0002_alter_helprequest_status_and_more.py` — Status field updates
3. `0003_payment_razorpay_order_id_and_more.py` — Adds Razorpay fields to Payment
4. `0004_alter_application_status.py` — Application status choices update

5. `0005_helprequest_request_type_helprequest_target_year.py` — Adds request metadata
6. `0006_helprequest_image.py` — Adds Cloudinary image field

**Chat:**

1. `0001_initial.py` — Creates `ChatMessage`
2. `0002_chatmessage_application.py` — Links messages to applications

---

## 12. Complete URL Routing Map

| URL Pattern | App | View Name | Access | Description |
|---|---|---|---|---|
| `/admin/` | Django | admin | Staff | Admin panel |
| `/` | accounts | `landing` | Public | Landing page |
| `/register/` | accounts | `register` | Public | Registration |
| `/login/` | accounts | `login` | Public | Login |
| `/logout/` | accounts | `logout` | Auth POST | Logout |
| `/verify-otp/` | accounts | `verify_otp` | Public | OTP verification |
| `/resend-verification/` | accounts | `resend_verification` | Public | Resend OTP |
| `/profile/` | accounts | `profile` | Auth | Own profile |
| `/profile/edit/` | accounts | `edit_profile` | Auth | Edit profile |
| `/profile/<username>/` | accounts | `user_profile` | Auth | Public profile |
| `/dashboard/` | work | `dashboard` | Auth | Dashboard |
| `/request/new/` | work | `create_help_request` | Auth | Create request |
| `/request/<pk>/` | work | `help_request_detail` | Auth | Request detail |
| `/browse/` | work | `browse_requests` | Auth | Browse requests |
| `/my-requests/` | work | `my_requests` | Auth | My requests |
| `/request/<pk>/apply/` | work | `apply_to_help` | Auth | Apply to help |
| `/application/<pk>/withdraw/` | work | `withdraw_application` | Auth | Withdraw app |

| URL Pattern | App | View Name | Access | Description |
|---|---|---|---|---|
| `/my-applications/` | work | `my_applications` | Auth | My applications |
| `/request/<pk>/resolve/` | work | `resolve_request` | Auth | Resolve request |
| `/request/<pk>/close/` | work | `close_request` | Auth | Close request |
| `/request/<pk>/payment/` | work | `payment_page` | Auth | Payment page |
| `/request/<pk>/payment/create-order/` | work | `create_razorpay_order` | Auth POST AJAX | Create Razorpay order |
| `/request/<pk>/payment/confirm/` | work | `confirm_payment` | Auth POST | Verify payment |
| `/request/<pk>/payment/receipt/` | work | `payment_receipt` | Auth | Payment receipt |
| `/notifications/` | work | `notifications` | Auth | Notifications |
| `/notifications/<pk>/read/` | work | `mark_notification_read` | Auth | Mark read |
| `/notifications/mark-all-read/` | work | `mark_all_read` | Auth | Mark all read |
| `/request/<pk>/chat/<app_pk>/` | chat | `chat_room` | Auth | Chat thread |
| `/request/<pk>/chat/<app_pk>/send/` | chat | `send_message` | Auth POST | Send message |
| `/request/<pk>/chat/<app_pk>/fetch/` | chat | `fetch_messages` | Auth GET AJAX | Poll messages |
| `/my-chats/` | chat | `my_chats` | Auth | Chat list |

# 13. Request / Response Lifecycle

## Typical Page Request

```
Browser Request
    → WhiteNoise (static files) or Django
    → SecurityMiddleware (headers, HTTPS)
    → SessionMiddleware (load session)
    → CSRFMiddleware (validate token)
    → AuthenticationMiddleware (load user)
    → MessageMiddleware (flash messages)
    → URL Resolver (nxthelp/urls.py)
        → App URL Resolver (accounts/work/chat/urls.py)
            → View Function
                → Database Query (ORM)
                → Business Logic
                → Template Rendering (DTL)
                    → base.html + child template
                        → Static CSS/JS
    → HttpResponse
```

## Payment Flow (Razorpay)

```
1. Poster visits /request/<pk>/payment/
   → Backend calculates amount from application budget
   → Renders template with Razorpay Key ID

2. Poster clicks "Pay Now"
   → JS calls /request/<pk>/payment/create-order/ (AJAX POST)
   → Backend creates Razorpay order
   → Backend creates Payment record (status: created)
   → Returns {order_id, amount, currency, payment_pk}

3. Razorpay Checkout Popup
   → Frontend opens Razorpay.js with order_id
   → User completes payment on Razorpay's UI
   → Razorpay returns {payment_id, order_id, signature}

4. Payment Confirmation
   → Frontend POSTs to /request/<pk>/payment/confirm/
   → Backend verifies signature with Razorpay API
   → On success: Payment.status = completed, HelpRequest.status = resolved
   → Redirects to receipt page
```

## Chat Message Flow

```
1. User sends message
   → POST to /request/<pk>/chat/<app_pk>/send/
   → Backend creates ChatMessage
   → Backend creates Notification for recipient
   → If AJAX: returns JSON; else redirects

2. Polling for new messages
   → Frontend polls /request/<pk>/chat/<app_pk>/fetch/?last_id=X
   → Backend returns messages with ID > last_id
   → Frontend appends to chat UI
```

---

# 14. Security Architecture

## Implemented Defenses

| Threat | Mitigation | Location |
|---|---|---|
| **CSRF** | Django's `CsrfViewMiddleware` + token in all forms/HTMX headers | `settings.py`, `base.html` |
| **XSS** | Django template auto-escaping + `SECURE_BROWSER_XSS_FILTER` | All templates, `settings.py` |
| **Clickjacking** | `XFrameOptionsMiddleware` with `DENY` | `settings.py` |
| **Open Redirect** | `url_has_allowed_host_and_scheme()` validation | `accounts/views.py` ( `login_view` ) |
| **Email Enumeration** | Generic messages for resend/invalid login | `accounts/views.py` |
| **IDOR (Chat)** | Ownership check ( `request.user in [poster, applicant]` ) | `chat/views.py` |
| **Payment Tampering** | Backend-calculated amount; signature verification | `work/views.py` ( `create_razorpay_order`, `confirm_payment` ) |
| **Session Hijacking** | `SESSION_COOKIE_HTTPONLY`, `SESSION_COOKIE_SECURE` (prod) | `settings.py` |
| **Password Security** | Django's built-in validators (length, common, numeric) | `settings.py` |
| **Inactive Users** | Cannot login; redirected to OTP verification | `accounts/views.py` |
| **Rate Limiting (OTP)** | Max 5 attempts + 60-second resend cooldown | `accounts/views.py` |

## Production Security Headers

- HSTS (1 year, includeSubDomains, preload)
- SSL redirect
- Secure cookies

- Content-Type nosniff
- X-Frame-Options: DENY

---

## 15. Deployment Configuration

### `railway.toml`

```
[build]
builder = "NIXPACKS"

[deploy]
startCommand = "python manage.py collectstatic --noinput && python manage.py migrate --noinput && gunicorn nxthe
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 10
```

**Deployment Flow:**
1. Railway detects Python app via Nixpacks.
2. Installs dependencies from `requirements.txt`.
3. Runs `collectstatic` to gather static files into `staticfiles/`.
4. Runs `migrate` to apply database migrations.
5. Starts Gunicorn bound to `$PORT`.

### WhiteNoise Static Serving

- `STATIC_ROOT = BASE_DIR / 'staticfiles'`
- `STATICFILES_STORAGE = 'whitenoise.storage.CompressedStaticFilesStorage'`
- WhiteNoise middleware is second in the middleware stack.

### Cloudinary Media Storage (Production)

- If `CLOUDINARY_CLOUD_NAME`, `API_KEY`, and `API_SECRET` are set:
- `DEFAULT_FILE_STORAGE` points to Cloudinary.
- All uploaded images go to Cloudinary's `request_images` folder.

---

# 16. Environment Variables Reference

## Required

| Variable | Purpose | Used In |
|---|---|---|
| `DJANGO_SECRET_KEY` | Cryptographic signing | `settings.py` |
| `DATABASE_URL` | Production database connection | `settings.py` |
| `RAZORPAY_KEY_ID` | Razorpay API key | `settings.py`, `work/views.py` |
| `RAZORPAY_KEY_SECRET` | Razorpay API secret | `settings.py`, `work/views.py` |

## Email / OTP

| Variable | Purpose | Default |
|---|---|---|
| `BREVO_API_KEY` | Brevo API for sending OTP emails | None |
| `EMAIL_HOST_USER` | SMTP username | None |
| `EMAIL_HOST_PASSWORD` | SMTP password | None |
| `DEFAULT_FROM_EMAIL` | From address | `NxtHelp <noreply@example.com>` |
| `EMAIL_BACKEND` | Django email backend | `smtp.EmailBackend` |

## Cloudinary (Optional)

| Variable | Purpose |
|---|---|
| `CLOUDINARY_CLOUD_NAME` | Cloudinary cloud name |
| `CLOUDINARY_API_KEY` | Cloudinary API key |
| `CLOUDINARY_API_SECRET` | Cloudinary API secret |

### Deployment

| Variable | Purpose |
| --- | --- |
| `DJANGO_DEBUG` | Enable debug mode ( `True` / `False` ) |
| `DJANGO_ALLOWED_HOSTS` | Comma-separated allowed hosts |
| `CSRF_TRUSTED_ORIGINS` | Comma-separated trusted origins |

---

## 17. Management Commands

### Built-in Django

```
python manage.py runserver          # Dev server
python manage.py migrate            # Apply migrations
python manage.py makemigrations     # Create new migrations
python manage.py collectstatic      # Gather static files
python manage.py createsuperuser    # Create admin user
python manage.py shell              # Django shell
```

### Custom Commands

```
python manage.py seed_categories    # Seed 8 default categories
```

---

## 18. Development Workflow Guide

### For New Developers

1. **Setup Environment:**
   `bash pip install -r requirements.txt # Create .env file with required variables`
2. **Database Setup:**
   `bash python manage.py migrate python manage.py seed_categories python manage.py createsuperuser`
3. **Run Development Server:**
   `bash python manage.py runserver`

4. **Making Changes:**
5. **Models:** Edit `models.py` → `makemigrations` → `migrate`
6. **Views:** Edit `views.py` (no restart needed with dev server)
7. **Templates:** Edit `.html` files (auto-reload)
8. **Static CSS:** Edit `style.css` (may need hard refresh)
9. **Adding a New URL:**
10. Add path to app's `urls.py`
11. Create view function in `views.py`
12. Create template if needed
13. Add tests
14. **Adding a New App:**
    `bash python manage.py startapp newapp`
15. Add `'newapp.apps.NewappConfig'` to `INSTALLED_APPS`
16. Create `urls.py` in new app
17. Include in `nxthelp/urls.py`

## Key Architectural Principles

- **No Role System:** Everyone is equal. Any user can post requests and apply to help.
- **Status-Driven Workflow:** `HelpRequest` statuses ( `open` → `in_progress` → `completed` / `resolved` / `closed` ) drive UI and permissions.
- **Application-Centric Chat:** Chat messages are tied to `Application` , not directly to `HelpRequest` . This allows multiple parallel conversations per request.
- **Backend-Trusted Payments:** Payment amounts are always calculated server-side from the accepted application's proposed budget.
- **Lazy Razorpay Initialization:** The Razorpay client is only created when first needed, preventing startup crashes in dev environments without payment keys.

---

*End of Architecture Guide*